SYNTOGRADE

Long-Running System Technical Analysis

Introduction

Taking over an existing software system requires a structured approach. Long-running applications often contain undocumented functionality, hidden dependencies, and architectural decisions that are not immediately visible from the outside.

Before implementing new features, we first focus on understanding the current system. This reduces the risk of breaking existing functionality and ensures that future development is built on a solid technical foundation.

Our technical analysis consists of four stages:

1. Environment Setup
2. Code & Architecture Review
3. Feature Strategy Evaluation
4. Risk Assessment & Mitigation

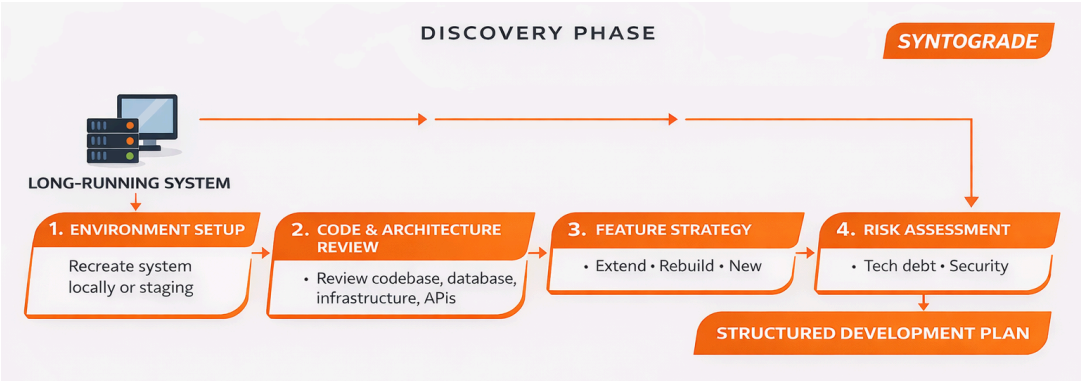
Each stage provides insights that guide the next step.

Assessment Approach

Our assessment combines automated analysis with manual architectural review.

Automated tools help identify issues such as outdated dependencies, potential security vulnerabilities, and code quality concerns. Manual review focuses on understanding the system architecture, infrastructure setup, and operational workflows.

This approach allows us to build an accurate understanding of the system before recommending any architectural changes.



SYNTOGRADE

System Discovery Process

Phase 1. Environment Setup

What we do

The first step is to reproduce the application in a controlled development environment.

Using the provided source code, database snapshot, and infrastructure information, we recreate the system locally or in a staging environment. This allows our engineers to run and inspect the application independently from the production system.

During this stage we typically:

- set up a development environment that mirrors the production stack
- restore the database snapshot provided by the client
- configure required dependencies and services
- connect the application to the database and supporting services
- verify that the system starts and functions correctly

If the application fails to run or requires undocumented configuration changes, we document those findings as part of the system understanding.

Why this step is important

Working directly on production systems is risky and makes proper analysis difficult. Reproducing the system in a controlled environment allows us to safely explore how the application behaves.

This step also confirms that we fully understand how the system is deployed and configured. In long-running systems, deployment processes are often undocumented, and recreating the environment helps uncover hidden dependencies or configuration requirements early in the process.

Without a working development environment, reliable analysis and safe development are not possible.

SYNTOGRADE

Phase 2. Code & Architecture Review

What we do

Once the system is running in a development environment, we perform a technical review of the application to understand how it is structured and how its components interact.

This review focuses on several areas.

First, we examine the **codebase structure** to understand how the application is organized, which frameworks are used, and how maintainable the code is.

Second, we review the **database schema**, including the relationships between tables, the structure of the data model, and any constraints or performance considerations.

Third, we analyze the **infrastructure and operational setup**, such as hosting environments, deployment procedures, and external services the application depends on.

Finally, we also identify **external dependencies** used by the system, including third-party libraries, APIs, and supporting services. Understanding these dependencies helps reveal potential risks such as unsupported libraries, deprecated APIs, or services that are critical to the system's operation.

Where possible, we review **the primary workflows** within the system to understand how it is used in practice. This helps ensure that future changes support existing operational processes and do not disrupt day-to-day usage.

Why this step is important

Long-running systems often contain architectural decisions and dependencies that are not obvious from documentation alone. Without a detailed review, development teams may unintentionally introduce changes that break existing functionality.

Understanding the architecture allows us to:

- estimate the complexity of future changes
- identify areas of technical debt
- detect potential security issues
- understand how different system components interact

This knowledge forms the foundation for making informed decisions about how new functionality should be implemented.

SYNTOGRADE

Phase 3. Feature Strategy Evaluation

What we do

After analyzing the system, we evaluate how the requested new features should be implemented.

The desired functionality is typically described through user stories or feature requests. Based on these requirements and the results of the architecture review, we determine the most appropriate development approach.

Our recommendation is based on the condition of the existing architecture, the complexity of the requested changes, and the long-term maintainability of the system. The goal is to balance short-term development efficiency with long-term system sustainability.

In most cases there are three possible strategies.

The first option is to **extend the existing codebase.** If the current architecture is stable and maintainable, we can build new features directly on top of the existing system.

The second option is to **gradually rebuild the application** while the existing system continues to operate. In this approach, a new version of the system is developed module by module, while the current application remains active.

The third option is to keep the existing application largely unchanged while introducing **new standalone applications.** New functionality is developed as separate components that integrate with the current system through APIs or other integration mechanisms.

For each option we evaluate technical feasibility, development complexity, and long-term maintainability.

Why this step is important

Choosing the wrong development strategy can create significant long-term problems.

Continuing to extend a fragile system may increase technical debt and make future development increasingly difficult. On the other hand, replacing a system unnecessarily can introduce unnecessary cost and migration risk.

By carefully evaluating the architecture and requirements, we ensure that the chosen strategy balances short-term delivery with long-term sustainability.

This step ensures that development decisions are based on technical evidence rather than assumptions.

SYNTOGRADE

Phase 4. Risk Assessment & Mitigation

What we do

Before development begins, we perform a structured risk assessment based on the insights gathered during the earlier stages.

Long-running systems often contain hidden complexities that only become visible during detailed analysis. We identify these risks and document how they might affect development or system stability.

Typical risks may include:

- undocumented business logic
- hidden integrations with external services
- fragile database structures
- security vulnerabilities
- deployment or infrastructure limitations

For each identified risk we define mitigation strategies. These strategies may include additional testing procedures, staged rollouts, monitoring improvements, or rollback plans.

If larger architectural changes or system replacement are considered, we also evaluate risks related to data migration, system downtime, and compatibility with existing integrations.

Why this step is important

Identifying risks early allows us to address them proactively rather than reacting to problems during development.

A structured risk assessment helps ensure that the project proceeds in a controlled and predictable manner. It also allows stakeholders to understand potential challenges and plan accordingly.

By defining mitigation strategies upfront, we reduce the likelihood of unexpected disruptions and improve the reliability of the development process.

SYNTOGRADE

Deliverables

After completing the technical analysis, we provide a summary of our findings.

Typical deliverables include:

- identification of technical risks and technical debt
- recommendation for the development approach
- high-level roadmap for implementing future changes

These findings form the basis for planning the next phase of development.

Limitations

Long-running systems often contain undocumented functionality or hidden dependencies that may only become visible during development. While the discovery phase aims to identify major risks and architectural constraints, additional insights may emerge as work progresses.

SYNTOGRADE

Initial Information Request

To begin analyzing the system, we only require a small amount of technical information. The goal of this request is simply to help us reproduce the application in a development environment.

If certain information is unavailable, that is completely fine. We can usually discover most details during the technical analysis.

1. Source Code

Access to the application source code allows us to understand how the system is structured.

Please provide one of the following:

- Access to the available Git repositories (highly recommended)
- A ZIP archive containing the codebase

2. Database Snapshot

To reproduce the system locally, we need a database dump.

If necessary:

- remove or anonymize personal data
- passwords can remain hashed

3. Running Environment (Optional but Helpful)

If a staging or test environment exists, access can significantly speed up the analysis.

Examples include:

- staging website login
- SSH access to a staging server
- instructions on how the system is typically deployed

Access to production is **not required**.

SYNTOGRADE

4. Technology Overview (If Known)

If available, a short overview of the system's technology stack helps us prepare the development environment.

Examples:

- programming language or framework
- database type
- major external services

If this information is unknown, we can determine it during the review.

5. External Services

If the application integrates with external services, please provide a short list.

Examples include:

- payment providers
- authentication providers
- email or messaging services
- external APIs

Detailed configuration is not required at this stage.

6. Known Issues (Optional)

If there are known technical issues or areas of concern within the system, a brief description can help guide the analysis.

Examples might include:

- performance problems
- components that are difficult to modify
- outdated libraries or services

This information is optional but helpful.

SYNTOGRADE

Typical Characteristics of Long-Running Systems

Software systems that have been running successfully for a longer period of time often evolve through multiple phases of development. New features are added, infrastructure changes, and different teams may contribute to the system over time.

Because of this natural evolution, it is common for systems to contain elements that are not immediately visible to new developers. Part of the discovery phase is simply understanding these aspects so future changes can be implemented safely.

Some examples of what we often encounter include the following.

Knowledge Held by Current Maintainers

Many operational details of a system are often known by the developers who have been maintaining it. This may include deployment procedures, infrastructure configuration, or historical decisions about how certain features were implemented.

This knowledge is extremely valuable during the transition and helps new developers understand the system more quickly.

Business Logic Embedded in the Application

In mature systems, important business rules are sometimes implemented directly in the application code. These rules may reflect years of operational experience and can be essential to how the system behaves in practice.

Understanding these rules helps ensure that future changes preserve the intended system behaviour.

Historical Architectural Decisions

Systems that have evolved over time often contain design decisions that were made to address specific requirements or constraints at the time. These decisions frequently make sense within the historical context of the system.

Part of the discovery process is understanding these decisions so that future development builds on them appropriately.

External Integrations and Supporting Services

Many systems interact with external services such as authentication providers, payment systems, messaging services, or internal tools. Identifying these integrations helps ensure that changes to the system continue to work smoothly with the surrounding ecosystem.

Operational Practices

Over time, teams often develop operational practices around a system, such as scheduled jobs, maintenance tasks, or monitoring procedures. Understanding these practices helps ensure the system continues to run reliably.